

ИНСТИТУТ БИЗНЕСА И ДИЗАЙНА
ОТЧЁТ О ПРАКТИЧЕСКОЙ РАБОТЕ / № 02

Интерактивный UI: state, события и hooks

Разработка мобильных приложений · БИInf-24

АВТОР	Телков Владимир	ПРЕПОДАВАТЕЛЬ	Саенко Я.Д.
ДАТА	16.09.2026	ГОРОД	Москва

01 / ПАСПОРТ ОТЧЁТА

Паспорт отчёта

Поле	Значение
Дисциплина	Разработка мобильных приложений
Практическая работа	№ 02 — Интерактивный UI: state, события и hooks
Студент	Телков Владимир Сергеевич
Группа	БИInf-24
Преподаватель	Саенко Я.Д.
Дата	16.09.2026
Связанный спринт	Sprint 2 — интерактивный локальный каталог
Ссылка на реквест с изменениями	https://github.com/Leladatan/study-mobile-project/pull/3
Тег	sprint-2

Цель работы

Добавить в каталог поиск, фильтр и выбор карточек, не сломав то, что было сделано в Sprint 1.

Задачи

- заменить ручной вывод карточек на `FlatList`;
- сделать управляемое поле поиска;
- добавить фильтр по региону и способ его сбросить;

- добавить выбор карточек со счётчиком;
- сделать задержку поиска через `useEffect`;
- вынести логику каталога в hook `useCatalog`;
- обновить README, backlog, CHANGELOG и ретроспективу.

02 / ВЫПОЛНЕНИЕ

Что сделано

Файл	Что изменилось
<code>src/hooks/useCatalog.ts</code>	новый hook с логикой каталога
<code>src/components/SearchField.tsx</code>	поле поиска
<code>src/components/RegionFilter.tsx</code>	фильтр по региону
<code>src/components/TopicCard.tsx</code>	проп <code>selected</code> и галочка
<code>src/screens/CatalogScreen.tsx</code>	<code>FlatList</code> , счётчики, пустое состояние
<code>docs/retrospective.md</code>	ретроспектива
<code>assets/sprint-2.png</code> , <code>assets/sprint-2.webm</code>	скриншот и видео

Ниже код показан в том виде, в каком он был влит в pull request #3. Что поменялось после, описано в конце раздела.

Hook `useCatalog`

Вся логика каталога лежит в `src/hooks/useCatalog.ts`. Экран только получает из него данные и функции и рисует интерфейс.

Состояние:

```
const [query, setQuery] = useState('');
const [appliedQuery, setAppliedQuery] = useState('');
const [regionId, setRegionId] = useState<RegionId | null>(null);
const [selectedIds, setSelectedIds] = useState<string[]>([]);
```

- `query` — текст в поле поиска;
- `appliedQuery` — текст, по которому фильтруется список;
- `regionId` — выбранный регион или `null`, если выбраны все;

- `selectedIds` — выбранные карточки.

Задержка поиска

```
useEffect(() => {
  const timer = setTimeout(() => setAppliedQuery(query), SEARCH_DELAY);
  return () => clearTimeout(timer);
}, [query]);
```

Когда меняется `query`, эффект ставит таймер на 250 мс. Если пользователь продолжает печатать, `cleanup` отменяет старый таймер, и список обновляется один раз — после паузы. Эффект зависит от `query`, а меняет `appliedQuery`, поэтому бесконечного цикла нет.

Поиск и фильтр

```
const visibleTopics = useMemo<Topic[]>(() => {
  const needle = appliedQuery.trim().toLowerCase();
  return TOPICS.filter((topic) => {
    if (regionId !== null && topic.regionId !== regionId) return false;
    if (needle.length === 0) return true;
    return (
      topic.title.toLowerCase().includes(needle) ||
      topic.capital.toLowerCase().includes(needle)
    );
  });
}, [appliedQuery, regionId]);
```

Запрос и названия переводятся в нижний регистр, поэтому `JAPAN` и `japan` работают одинаково. Пустой запрос возвращает все страны. Отфильтрованный список не хранится в `state`, а каждый раз считается из запроса и региона — так он не может разойтись с тем, что выбрал пользователь.

Выбор карточек

```
const toggleSelection = useCallback((topic: Topic) => {
  setSelectedIds((previous) =>
    previous.includes(topic.id)
      ? previous.filter((id) => id !== topic.id)
      : [...previous, topic.id],
  );
}, []);
```

Массив не меняется напрямую. При добавлении создаётся новый массив через `[...previous, id]`, при удалении — через `filter`. Перед добавлением проверяется `includes`, поэтому дубликатов нет. Счётчик «Выбрано» — это длина массива.

Поле поиска

Основные props поля в `SearchField.tsx`:

```
<TextInput
  style={styles.input}
  value={value}
  onChangeText={onChange}
  placeholder={placeholder ?? 'Страна или столица'}
/>
```

Поле управляемое: текст берётся из `state`, а при вводе `state` обновляется через `onChangeText`. Поэтому кнопка «Сбросить фильтры» может очистить поле из кода.

Фильтр по региону

В ленте есть чип «Все регионы». Повторное нажатие на выбранный регион снимает фильтр:

```
onPress={() => onSelect(selected === region.id ? null : region.id)}
```

Список

```
const renderItem = useCallback<ListRenderItem<Topic>>(  
  ({ item, index }) => (  
    <TopicCard  
      topic={item}  
      onPress={toggleSelection}  
      featured={index === 0 && !filtersActive}  
      selected={isSelected(item)}  
    />  
  ),  
  [toggleSelection, isSelected, filtersActive],  
)  
;  
  
const keyExtractor = useCallback((item: Topic) => item.id, []);
```

`FlatList` рисует только те карточки, которые видны на экране. Ключ строки — `item.id`, а не индекс: после фильтрации индексы меняются, а `id`

остаётся тем же. Первая карточка выделяется только тогда, когда фильтры не активны, иначе выделение прыгало бы при каждом поиске.

Если ничего не найдено, `FlatList` показывает `ListEmptyComponent` — текст с причиной и кнопку «Сбросить фильтры».

Проверка

Проверка	Результат
<code>npm run typecheck</code>	без ошибок
<code>npx expo-doctor</code>	21/21
Установка с нуля (<code>npm ci</code>)	без ошибок
Сборка под Android (<code>npx expo export</code>)	бандл собран
<code>push</code> , <code>splice</code> , ключ по индексу, <code>any</code>	нет
Полный каталог	12 карточек
Поиск <code>JAPAN</code>	«1 из 12»
Пустой запрос	все 12 карточек
Поиск по столице <code>ottawa</code>	«1 из 12», Canada
Регион «Европа и Центральная Азия»	«4 из 12»
<code>japan</code> при фильтре по Европе	«Ничего не найдено»
Кнопка сброса	снова 12 карточек
Выбор Japan и Brazil	«Выбрано: 2»
Повторное нажатие на Japan	«Выбрано: 1»
Быстрый ввод, затем <code>united</code>	«2 из 12»
Смена фильтра при выбранных карточках	отметки сохраняются

Сценарии проверены в браузере автоматическим скриптом, который вводил текст и нажимал на элементы.

Как показать на приёмке

Шаг	Что сделать
Витрина Sprint 1 на месте	<code>npm start</code> — 12 карточек, у Canada заглушка
Поиск	<code>JAPAN</code> и <code>japan</code> дают одно и то же, <code>zzz</code> — пустое состояние

Шаг	Что сделать
Фильтр и сброс	выбрать регион, потом «Все регионы»
Выбор	отметить и снять две карточки, смотреть на «Выбрано»
Код	CatalogScreen.tsx, SearchField.tsx, toggleSelection в useCatalog.ts
useEffect и cleanup	начало useCatalog.ts
Pull request, CHANGELOG, тег, видео	PR #3, [sprint-2], sprint-2, assets/sprint-2.webm

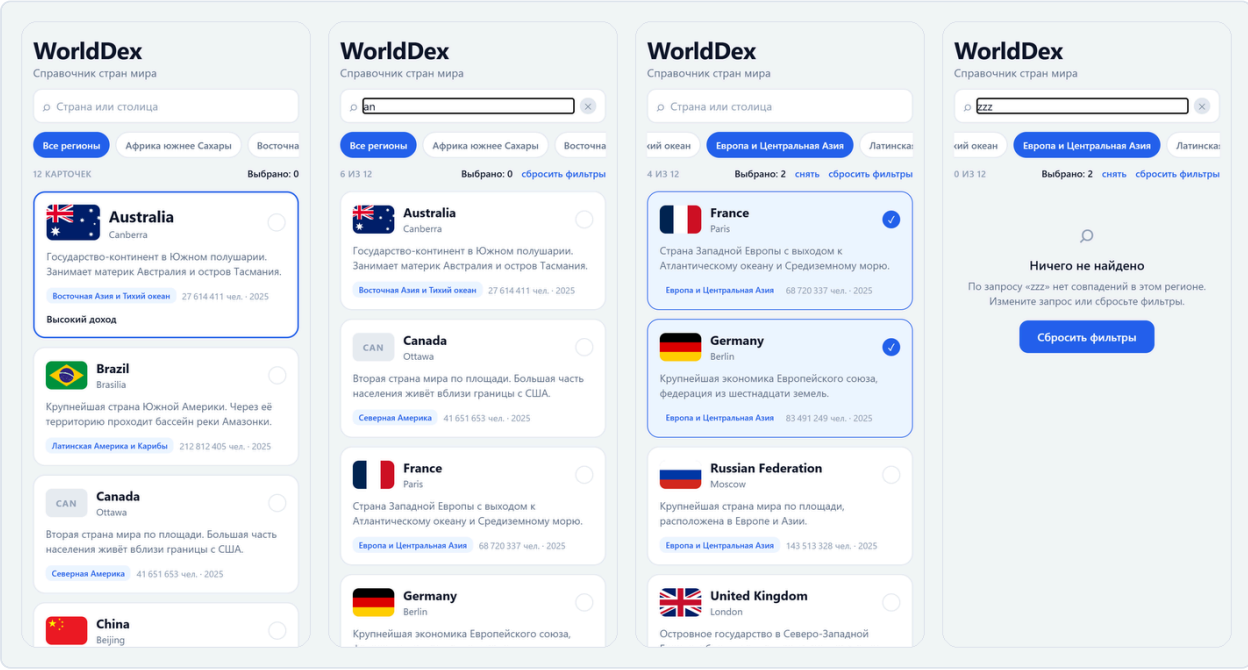


Рисунок 1 — Слева направо: полный каталог; поиск `an`; фильтр по Европе с двумя выбранными карточками; пустое состояние. Видео тех же действий — `assets/sprint-2.webm`

Доработки после слияния

После pull request #3 в проект внесены изменения:

- подключён ESLint (`npm run lint`), его требует Definition of Done;
- в README дописаны описание hook и раздел «Как проверить»;
- `regionId` переименован в `regionCode`, вместо union-типов используются `enum`;
- выбранные `id` хранятся в `Set` вместо массива;

- `TopicCard` передаёт в `onPress` только `id`;
- `TopicCard`, `SearchField` и `RegionFilter` обернуты в мемо;
- `expo` обновлён до 57.0.24, `@expo/metro-runtime` — до 57.0.16.

Выбор после перехода на `Set`:

```
const toggleSelection = useCallback((id: string) => {
  setSelectedIds((previous) => {
    const next = new Set(previous);
    if (next.has(id)) {
      next.delete(id);
    } else {
      next.add(id);
    }
    return next;
  });
}, []);
```

`add` и `delete` вызываются у копии `next`, а старый `previous` не меняется, поэтому `state` по-прежнему не мутируется. Проверка «выбрана ли карточка» теперь `selectedIds.has(id)` — она не перебирает весь список, как `includes`.

После доработок все проверки проходят: типы, ESLint (0 ошибок и 0 предупреждений), `expo-doctor` 21/21 и те же сценарии в браузере.

03 / РЕЗУЛЬТАТЫ

Результат

Каталог стал интерактивным: работают поиск, фильтр по региону, выбор карточек со счётчиком и пустое состояние. Логика вынесена в `useCatalog`, экран отвечает только за отображение. Всё, что было в Sprint 1, работает как раньше.

Таблица 1 — Критерии приёмки

Критерий	Как выполнен	Статус
<code>FlatList</code> с типизированными <code>data</code> и <code>renderItem</code> , ключ — <code>id</code>	<code>ListRenderItem<Topic></code> , <code>keyExtractor</code> → <code>item.id</code>	выполнено
<code>TextInput</code> управляется через <code>value</code> и <code>onChangeText</code>	<code>value={value}</code> , <code>onChangeText={onChange}</code>	выполнено

Критерий	Как выполнен	Статус
Поиск без учёта регистра, пустой запрос — весь каталог	<code>toLowerCase</code> , пустой запрос → 12 карточек	выполнено
Фильтр работает и сбрасывается, пустой результат объяснён	фильтр по региону, «Все регионы», пустое состояние	выполнено
Выбор иммутабельный, без дублей, счётчик верный	новый массив при каждом изменении, проверка <code>includes</code>	выполнено
Hooks вызываются на верхнем уровне	все вызовы в начале <code>useCatalog</code> и <code>CatalogScreen</code>	выполнено
<code>useEffect</code> с зависимостями и <code>cleanup</code>	<code>[query]</code> и <code>clearTimeout</code>	выполнено
Hook отделяет логику, Sprint 1 не сломан	экран не импортирует данные, карточки работают как раньше	выполнено

Таблица 2 — Ограничения

Ограничение	Статус
Не мутировать state через <code>push</code> , <code>splice</code> и присваивание	соблюдено
Не использовать индекс как ключ	соблюдено
Не вызывать hooks в условиях, циклах и после <code>return</code>	соблюдено
Не делать эффект, который меняет свою зависимость	соблюдено
Не подключать API, Redux, базу данных и навигацию	соблюдено

Таблица 3 — Definition of Done

Пункт	Статус
Поиск, фильтр, выбор и пустое состояние работают на целевой платформе	проверено в браузере и сборкой под Android; на телефоне — на защите
Проверка типов, lint и другие проверки проходят	выполнено; ESLint подключён после слияния
Нет мутации state и нарушений правил hooks	выполнено
Изменения прошли через pull request	выполнено, PR #3

Пункт	Статус
README и CHANGELOG описывают каталог и способ проверки	выполнено; раздел «Как проверить» дописан после слияния
Есть тег <code>sprint-2</code> и демонстрация	выполнено, видео и скриншот
В ретроспективе есть результат, затруднение, долг и следующий шаг	выполнено

Проблемы и решения

1. Список обновлялся бы на каждую букву. Если фильтровать по тексту в поле напрямую, при вводе `japan` список пересчитается пять раз. Решение — два состояния: `query` для поля и `appliedQuery` для списка с задержкой 250 мс.

2. Риск бесконечного цикла. Эффект, который меняет свою же зависимость, запускает сам себя. Здесь эффект зависит от `query`, а меняет `appliedQuery`.

3. Выделенная карточка прыгала бы. При поиске первой в списке может оказаться любая страна. Поэтому `featured` передаётся только без активных фильтров.

4. Тег поставлен до `git pull`. Pull request был влит, но локальный `main` не обновлён, и тег `sprint-2` встал на коммит Sprint 1. Тег перенесён командой `git tag -f`.

5. README обновился не полностью. Часть правок не применилась, и в PR #3 README описывал экран из Sprint 1. Описание и раздел «Как проверить» дописаны после слияния.

6. Не было линтера. К слиянию в проекте не было ESLint, хотя Definition of Done его требует. Он подключён командой `npm expo lint`, ошибок нет.

Ретроспектива

- **Получилось:** логика в одном hook, экран остался простым, карточку из Sprint 1 почти не пришлось менять.
- **Мешало:** задержку поиска нужно было сделать аккуратно; при публикации снова ошибка с тегом.

- **Долг:** выбор не сохраняется после перезапуска приложения.
- **Дальше:** загрузка данных из World Bank API в Sprint 3.

Два спринта вместе

	Sprint 1	Sprint 2
Сделано	тип <code>Topic</code> , карточка, экран	поиск, фильтр, выбор, <code>useCatalog</code>
Трудность	backlog не совпал с методичкой	задержка поиска без лишних таймеров
Долг	данные в коде, <code>ScrollView</code>	выбор не сохраняется
Ошибка публикации	тег до pull request	тег до <code>git pull</code>

04 / ЗАВЕРШЕНИЕ

Выводы

Каталог стал интерактивным: поиск, фильтр, выбор и пустое состояние работают, логика отделена от экрана. Все критерии приёмки выполнены, ограничения соблюдены. Компоненты из Sprint 1 удалось использовать почти без изменений, значит в прошлом спринте они были сделаны правильно.

05 / КОНТРОЛЬНЫЕ ВОПРОСЫ

Контрольный вопрос 1

Чем `state` отличается от обычной локальной переменной компонента?

Обычная переменная создаётся заново при каждой перерисовке и не обновляет экран, когда меняется. State React хранит между перерисовками, а вызов функции-сеттера перерисовывает компонент. Если бы запрос хранился в `let query = ''`, текст в поле пропал бы после каждой перерисовки.

Контрольный вопрос 2

Почему `TextInput` называется управляемым компонентом?

Потому что его значение задаёт React state, а не само поле. `SearchField` получает `value` из `query`, а при вводе вызывает `onChangeText`, который обновляет state. Благодаря этому поле можно очистить из кода, например кнопкой «Сбросить фильтры».

Контрольный вопрос 3

Зачем `FlatList` нужен стабильный `keyExtractor`?

По ключу React понимает, какая строка какая между перерисовками. Если ключ — индекс, после фильтрации одна страна получит ключ другой, и React перепутает строки. Код страны `item.id` не меняется, поэтому каждая карточка остаётся на своём месте.

Контрольный вопрос 4

Почему `props` и `state` рассматриваются как неизменяемые снимки?

React решает, нужно ли перерисовать экран, сравнивая ссылки на старое и новое значение. Если изменить массив через `push`, ссылка останется той же, и React может не заметить изменение. Поэтому создаётся новый массив через `[...previous, id]` или новый `Set` через `new Set(previous)`.

Контрольный вопрос 5

Какие два ограничения определяют допустимое место вызова `hooks`?

Первое: `hooks` вызываются только на верхнем уровне — не в условиях, циклах, вложенных функциях и не после `return`. Второе: только в компонентах или в других `hooks`. React отличает один `useState` от другого по порядку вызовов, и если порядок изменится, значения перепутаются.

Контрольный вопрос 6

Когда нужен `useEffect` и зачем ему `cleanup`?

`useEffect` нужен, чтобы работать с тем, что находится вне React: таймерами, подписками, запросами. В каталоге он ставит таймер для задержки поиска. `Cleanup` отменяет старый таймер перед новым запуском и при закрытии экрана, иначе при быстром вводе остались бы лишние таймеры.

Контрольный вопрос 7

Как пользовательский hook отделяет логику от отображения?

В `useCatalog` лежат state, эффект, поиск, фильтр и выбор. Наружу он отдаёт готовые данные и функции. `CatalogScreen` не знает, как работает фильтрация, и только рисует интерфейс. Когда данные начнут приходить из API, поменяется hook, а экран останется прежним.

Источники

1. React Native. FlatList. <https://reactnative.dev/docs/flatlist>
2. React Native. TextInput. <https://reactnative.dev/docs/textinput>
3. React. useState. <https://react.dev/reference/react/useState>
4. React. useEffect. <https://react.dev/reference/react/useEffect>
5. React. Rules of Hooks. <https://react.dev/reference/rules/rules-of-hooks>
6. React. State as a Snapshot. <https://react.dev/learn/state-as-a-snapshot>
7. React. Updating Arrays in State. <https://react.dev/learn/updating-arrays-in-state>
8. React. Reusing Logic with Custom Hooks. <https://react.dev/learn/reusing-logic-with-custom-hooks>

Дата обращения ко всем источникам: 16.09.2026.